ChatGPT

Agentic Debugging Literature Review

1. Executive Summary

This review finds that the largest body of mature work relevant to an **agentic debugging system** comes from four adjacent traditions rather than from one unified field: classical debugging and diagnosis, fault localization, automated program repair, and repository-level LLM software agents. Those traditions overlap, but they are not equivalent. In particular, most repository-level "software engineering agents" do **not** perform debugger-mediated runtime diagnosis; they usually operate over repository files, shell commands, test execution, and issue descriptions. The clearest currently verified exemplar of true debugger-controlling LLM behavior is **ChatDBG**, which integrates with **Pdb, GDB, and LLDB** and allows the model to inspect program state and navigate stacks while answering natural-language debugging questions. By contrast, systems such as **SWE-Agent**, **AutoCodeRover**, **Agentless**, and the historical **OpenHands/CodeAct** line are best understood as repository-level repair agents with varying degrees of tool use, retrieval, and test-feedback loops, but generally **without** first-class interactive debugger control. [1]

The literature also shows a persistent conceptual gap between **fault localization** and **root-cause analysis**. Most fault localization methods output a ranked list of suspicious program elements, commonly by suspiciousness score or learned ranking; that is useful, but it is not the same as explaining the causal chain that made the failure occur. This distinction is explicit in work such as Whyline, statistical debugging, and more recent LLM fault-localization systems that often provide explanations but are usually evaluated on rank-based localization metrics rather than on independently verified causal explanations. Claims that a system "found the root cause" are therefore much stronger than typical FL metrics alone justify. [2]

A second major finding is that **dynamic information matters**, but the field still underuses it in LLM-based repair. Classical FL and APR already exploited tests, coverage, execution spectra, mutation signals, slices, and symbolic constraints. Yet most modern LLM repair systems still rely primarily on static repository context and iterative test feedback rather than debugger-mediated state inspection. Recent work such as **Debug2Fix** and **debug-gym** explicitly argues that coding agents are deprived of rich runtime information and shows renewed interest in interactive debugging environments, but both are recent and not yet as established as the repository-level repair literature. [3]

For a **first realistic research prototype**, the literature most strongly supports a **Python-first** design centered on **Pdb**, failing tests or reproducible crashes, repository-aware code retrieval, structured runtime-state extraction, patch proposal, automated validation, and limited repair revision. Python is attractive because BugsInPy gives a real-bug benchmark, Pdb is standard and scriptable, ChatDBG already demonstrates debugger integration in Python, and Python-based SWE-style benchmarks and agents provide abundant comparison baselines. The strongest near-term scientific contribution would not be "yet another SWE-bench agent," but rather a system that demonstrates that **interactive debugger access provides information unavailable to static repository agents**, and that this information measurably improves localization, explanation, or repair under controlled evaluation. [4]

Coverage Map

Topic	Number of sources found	Full-text sources	Partial or abstract sources	Foundational coverage	Recent coverage	Confidence that coverage is sufficient
Debugging foundations	8	5	3	Strong	Moderate	Moderate
Automated debugging	9	6	3	Strong	Moderate	Moderate
Fault localization	15	10	5	Strong	Strong	Strong
Root-cause analysis	8	5	3	Moderate	Strong	Moderate
Automated program repair	16	10	6	Strong	Strong	Strong
LLM-based debugging and repair	18	10	8	Moderate	Strong	Strong
Agentic SE agents	14	8	6	Moderate	Strong	Strong
Interactive debugger-using agents	7	5	2	Limited	Strong	Moderate
Datasets and benchmarks	14	8	6	Strong	Strong	Strong

The weakest area is **software root-cause analysis for code-level debugging** as distinct from fault localization and from cloud/telemetry RCA. There is meaningful adjacent work, but the code-debugging literature still evaluates location ranking much more often than true causal diagnosis. [5]

2. Research Scope and Method

This review used English-language searches and prioritized primary papers, benchmark websites, official repositories, and conference proceedings. The most heavily relied-on primary sources were conference PDFs or arXiv HTML/PDF pages for **ChatDBG**, **SWE-Agent**, **OpenHands**, **AutoCodeRover**, **Agentless**, **RepairAgent**, **InferFix**, **ChatRepair**, **DebugBench**, **debug-gym**, **Defects4J**, **BugsInPy**, **QuixBugs**, and survey or synthesis papers on APR and fault localization. Official benchmark and repository pages were used mainly for implementation status, licensing, dataset composition, and version drift relative to papers. [6]

The review deliberately separated three evidence layers. **First**, historical and foundational evidence for what debugging, fault localization, and APR are. **Second**, system-specific evidence for what named systems actually do. **Third**, interpretation for project design implications. When a capability was not directly supported by a paper or official artifact, it was marked **Unclear**, **Partial**, or **Could not verify** rather than inferred optimistically. That is especially important for claims about debugger integration, runtime-variable access, multi-agent structure, and benchmark performance. [7]

The collection strategy also accounted for version drift. For several systems, the repository in 2026 differs materially from the paper-era artifact. For example, the current **SWE-agent** repository explicitly says current development has shifted to **mini-SWE-agent**; the current **OpenHands** repository emphasizes **Agent Canvas** and says the agent and server code have moved to a different repository; the **AutoCodeRover** and **Agentless** repositories report more recent performance numbers than their original papers. Those later repository claims were treated as implementation-status evidence, not as substitutes for peer-reviewed reported results. [8]

3. Source Access and Verification Policy

Every source below is classified with one of the required labels:

- **FULL_TEXT_READ**: used sparingly; only when the whole paper or document was inspected closely enough to justify full-text claims.
- **SUBSTANTIAL_SECTIONS_READ**: the main category for opened PDFs and HTML papers whose abstract, method, architecture, evaluation, and relevant tables or sections were inspected.
- **ABSTRACT_ONLY**: used when only abstract pages or arXiv abstract pages were inspected.
- **METADATA_ONLY**: used for repository landing pages, benchmark metadata pages, or discovery pages where only metadata-like sections were used.
- **INACCESSIBLE**: used when the source appeared relevant but the full text was not accessible or was not verifiably read.

In this report, most scientific claims rely on **SUBSTANTIAL_SECTIONS_READ** sources. Claims requiring detailed results, tool design, or evaluation methodology were not grounded in abstract-only sources. Search-result snippets were not used as scientific evidence where a paper or opened official page was available. [9]

4. Historical Development of the Field

Modern agentic debugging builds on a long arc of work. The classical era focused on breakpoint debuggers and trace inspection; later research emphasized higher-level questions such as “why did” and “why didn’t” a behavior happen, as in **Whyline**. Parallel work on **statistical debugging**, **dynamic invariant detection**, **program slicing**, and **delta debugging** pushed toward partial automation of diagnosis. A distinct line then emerged around **fault localization**, which treated debugging as a ranking problem over statements or methods, followed by **automated program repair**, which treated fixing as search or synthesis under tests. More recently, **neural and LLM-based** approaches replaced or augmented handcrafted repair spaces and ranking features, and repository-level **tool-using agents** began solving issue-resolution benchmarks such as SWE-bench. Only the newest layer adds genuine interactive debugger integration back into the loop. [10]

A useful historical simplification is therefore:

manual debugging → automated diagnosis aids → fault localization → automated repair → learning-based repair → LLM static repair → repository-level software agents → debugger-using agents.

That progression is real, but it is not a sequence of strict replacements. Newer work often revisits older signals: **AutoCodeRover** reuses SBFL when tests are available; **Agentless** still decomposes the task into localization, repair, and validation; **Debug2Fix** explicitly argues for bringing debugger information back into agents because shell-and-test loops omit runtime evidence familiar to human debuggers. [11]

5. Core Concepts and Terminology

Testing determines whether specified behaviors hold for selected inputs; **debugging** begins after a failure or suspect behavior has been observed and aims to explain and fix it. **Validation** asks whether the system satisfies intended needs; **verification** asks whether it satisfies formalized requirements. **Monitoring** and **observability** expose runtime signals such as logs, traces, and metrics, but they do not by themselves localize or repair faults. **Static analysis** reasons without executing the target program; **dynamic analysis** depends on observed executions or states. These distinctions matter because many recent LLM systems are called “debugging” systems even when they only perform static code analysis or iterative repair from test failures. [12]

A **fault** is the underlying defect in code; a **failure** is an observed incorrect behavior; a **root cause** is the causally responsible mechanism or condition that produced the failure in context. **Fault localization** usually ranks suspicious code elements. **Automated program repair** then explores a **patch space**, often guided by a **fault space**: the set of candidate locations to edit. A patch that passes the available tests is commonly called **plausible**, but it is not necessarily semantically correct; the literature repeatedly warns that weak tests permit **patch overfitting**. [13]

6. Traditional Debugging

Traditional debugging is an **iterative diagnostic process** in which developers update a mental model of the system, gather evidence, generate hypotheses, run experiments, and repeat until the failure mechanism is understood well enough to fix. A recent grounded-theory study of professional practice characterizes debugging exactly in those terms and reports that developers alternate between navigation and execution strategies while switching between forward and backward tracing modes of reasoning. [14]

Why manual debugging is difficult has been documented for years. **Whyline** argued that developers must translate questions about observed behavior into speculative questions about code, and that their initial guesses are often wrong; it also stressed that tools frequently require the developer to name an allegedly relevant line or variable before the tool can help, which makes them vulnerable to “garbage in, garbage out.” Separate empirical work on software maintenance found large navigational overhead in seeking, relating, and collecting relevant information. Together, these findings explain why stack inspection, breakpoint placement, stepping, variable inspection, logs, and reproduction attempts are useful but still cognitively expensive. [15]

Breakpoints, stepping, stack inspection, local-variable inspection, and replaying failures remain central because they expose **program state**. That is precisely the information many newer static or repository-level LLM systems lack. The human debugging literature therefore strongly supports giving an automated agent

access not just to code and failing tests, but also to stateful runtime evidence and a mechanism for controlled information-seeking. [16]

7. Automated Debugging

Historically, “automated debugging” has meant partial automation of diagnosis rather than end-to-end autonomous bug fixing. **Statistical debugging** instruments programs, gathers execution reports, and identifies predicates associated with particular bugs; **Whyline** automatically answered selected causal questions about behavior using static and dynamic slicing; **Daikon** inferred likely invariants from executions; and slicing-based methods reduced the search space around relevant dependencies. These systems automate evidence collection and explanation aids, but they usually do not autonomously commit a patch and validate it against a full repository workflow. [17]

The important technical families are still recognizable today: statistical or predicate-based diagnosis, coverage-based ranking, dynamic slicing, invariant-based diagnosis, model-based or causal diagnosis, and, later, repair-oriented variants that combine localization with candidate synthesis. Their assumptions differ widely. Some require passing and failing tests; some need reproducible crashes; some need instrumentation; some assume a reasonably bounded search space; and some depend on executable specifications or strong test suites. Their strengths and weaknesses likewise vary: dynamic methods can exploit concrete failures, but they are limited by test quality and reproduction fidelity; static and test-free methods cover more scenarios, but they often have weaker guarantees about causal relevance. [18]

8. Fault Localization

Fault localization is the most mature precursor to an agentic debugger. Classical **spectrum-based fault localization** computes suspiciousness scores from passing and failing test coverage. The 2022 IEEE Access survey gives a clean operational description: elements are ranked by suspiciousness; ties are common; and practical evaluation often focuses on how early faulty elements appear in the list. It also summarizes core metrics such as **EXAM**, the percentage of statements that must be examined before the real fault is found, and **Top-N** categories such as Top-1, Top-3, Top-5, and Top-10. [19]

The main technique families are:

- **Spectrum-based / statistical FL**: Tarantula, Ochiai, DStar-like measures, and related formulas over test spectra. These methods are efficient and benchmark-friendly, but they rank suspicious locations rather than explaining causal mechanisms. [20]
- **Predicate-based and statistical debugging**: exemplified by Liblit et al., which isolates predictors associated with distinct bugs from many executions. This moves closer to bug circumstances and failure modes, but still typically identifies correlates and predictors more than full root causes. [21]
- **Slicing-based FL**: uses static or dynamic dependence slices to restrict candidate code. Dynamic slicing can retain high recall for true faults, but can still be large and execution-specific. [22]
- **Mutation-based FL**: systems such as **MUSE**, **Metallaxis-FL**, and **SIMFL** exploit mutant behavior to rank suspicious locations, often improving ranking quality but at substantial computational cost. **SIMFL** is specifically notable for amortizing mutation cost ahead of time. [23]
- **Invariant-based FL**: dynamic invariant detection can expose violated assumptions or unexpected state relationships, which is more explanatory, though not by itself a full localization solution. [24]

- **Learning-based / neural FL**: MLFL systems and newer LLM-based approaches learn patterns of faultiness from code, coverage, or execution artifacts. **LLMAO** showed that LLM-derived representations can localize faults even without test coverage information, but that setting is still localization, not debugger-style diagnosis. [25]
- **Repository-level LLM FL**: systems such as **AutoFL**, **AgentFL**, and **LLM4FL** use tool calls, repository navigation, and sometimes multi-agent structures to rank suspicious methods in larger repositories. They often add explanations, but are still evaluated primarily on **method-level accuracy** or Top-N metrics. [26]

A central caution is that almost all FL metrics are **ranking metrics**, not causal-proof metrics. Top-N accuracy, MRR, and EXAM tell you how quickly the true bug appears in a ranked list. They do **not** establish that the system understood the underlying reason the bug exists. That distinction becomes especially important for LLM-generated explanations, which may be fluent even when unsupported. [27]

9. Root-Cause Analysis

Root-cause analysis is stronger than localization. A precise root-cause claim normally requires evidence that the identified mechanism or condition **causally produced** the failure and that altering that mechanism would prevent the failure under the relevant conditions. In code debugging, this typically requires some combination of control-flow reasoning, data-flow reasoning, execution history, state inspection, or counterfactual reasoning about what would have happened if a key value or path differed. Whyline is important here because it explicitly frames diagnosis around explanations of observed or missing behavior rather than mere suspicious ranking. Statistical debugging similarly tries to identify failure-associated predicates and circumstances, again moving beyond plain location ranking. [28]

However, the literature also shows that reliable evaluation of root-cause correctness is underdeveloped. In software quality assurance more broadly, a 2024 systematic review reports that causal reasoning is increasingly used in SQA and that fault localization is one of the main application areas, but maturity varies and many methods remain domain-specific. In cloud and telemetry settings, **OpenRCA** offers a benchmark for root-cause identification over logs, metrics, and traces, but that benchmark studies operational RCA more than source-code debugging. It is relevant as an evaluation idea, not as a replacement for code-debugging benchmarks. [29]

For an agentic debugger project, the literature supports a conservative standard: claim **root-cause analysis** only when the system can show a concrete causal narrative tied to runtime evidence, such as stack frames, variable states, condition values, or control dependencies, and when that narrative is separately checked by humans or by known ground truth. Ranking the correct line is useful but insufficient by itself. [30]

10. Automated Program Repair

Automated program repair has a long and well-structured literature. The CACM overview by Le Goues, Pradel, and Roychoudhury distinguishes **heuristic repair**, which explores a syntactic search space and validates candidates against tests, from **constraint-based repair**, which builds repair constraints and synthesizes code satisfying them. That paper also places FL centrally in the repair pipeline. [31]

Important seminal lines include:

- **Generate-and-validate / search-based repair**: **GenProg** and later **RSRepair** search a patch space using candidate transformations and tests as the acceptance criterion. The literature subsequently showed that simpler random search can perform surprisingly strongly and that search algorithms matter less than search-space structure in some cases. [32]
- **Template-based repair**: **PAR** learned fix patterns from human-written patches and applied them to real bugs. This line strongly influenced later template and mined-pattern systems. [33]
- **Constraint-based / semantic repair**: **SemFix** builds symbolic constraints from tests and synthesizes expressions satisfying them. Later work such as **SPR** and **Angelix** refined condition synthesis and repair search. [34]
- **Production-scale repair**: systems from industry such as **InferFix** and large-scale static-analysis-integrated repair show how bug detection, localization, categorization, fix generation, and validation can be deployed in CI-style workflows, but these are usually static-analysis-first rather than debugger-first. [35]

Two lessons from APR are crucial for agentic debugging. First, **plausible is not correct**: passing a test suite does not prove semantic correctness, and patch overfitting remains a central challenge. Second, repair quality depends heavily on the **fault space** and the **patch space**. Good localization and good candidate constraints often matter more than raw model strength alone. [36]

11. Learning-Based and Neural Program Repair

Learning-based APR moved from handcrafted transformations to data-driven prediction of edits or fix patterns. Early neural approaches treated bug fixing as seq2seq translation over buggy and fixed code. Later work incorporated richer retrieval and metadata. For example, **InferFix** combines a retriever with a finetuned generator and pairs them with the **Infer** static analyzer for bug detection, localization, and categorization; it reports strong top-1 accuracy on Microsoft's Java and C# static-analysis bug datasets. [35]

By 2024–2025, the literature differentiated at least three neural subfamilies: **prompted LLM repair**, **retrieval-augmented repair**, and **agentic repair**. **ChatRepair** is a strong example of conversational repair that interleaves candidate generation with immediate test-failure feedback; **RepairAgent** turns the LLM into an autonomous tool-using repair agent on Defects4J; and **AutoCodeRover** and **Agentless** push toward repository-level issue solving. These systems show that modern repair is no longer just one-shot generation, but they also show that additional tool use does not automatically imply dynamic debugging in the debugger sense. [37]

12. LLM-Based Debugging

The recent LLM-based debugging landscape is heterogeneous. At least six distinct modes appear in practice:

1. **Static explanation and repair from code only**.
2. **Error-message-assisted repair**.
3. **Test-feedback-driven iterative repair**.
4. **Repository-level tool-using issue resolution**.
5. **Runtime-information-assisted self-debugging**.

6. **Interactive debugger-controlling debugging**.

Conflating these modes causes much of the present literature confusion. For example, **LLMAO** does test-free fault localization from code; **AutoFL** uses failing tests and repository navigation to localize methods and explain them; **SWE-Agent** uses shell commands, editing, and tests to resolve issues; **LDB** and related work use runtime execution information to refine generated code; and **ChatDBG** directly controls debuggers. These all matter for agentic debugging, but they occupy different points on a static-to-dynamic spectrum. [38]

A strong recent benchmark-level indicator of the importance of debugging-specific evaluation is **DebugBench**, which was created because prior evaluations of LLM debugging were limited by leakage risk, scale, and bug variety. DebugBench comprises **4,253** instances across **C++, Java, and Python**, covering **four major bug categories** and **18 minor types**. Its findings also caution that runtime feedback is not uniformly beneficial; simply feeding back execution information is not enough unless the model or agent can use it effectively. [39]

13. Agentic and Tool-Using Debugging

Repository-level agents are now a major comparison class for any agentic debugger project. **SWE-Agent** introduced the idea of an **agent-computer interface** tailored to LMs, arguing that better interfaces can materially improve software-agent performance without changing model weights. Its paper shows a large gap between simple retrieval baselines and a shell-based agent loop on SWE-bench. [40]

AutoCodeRover takes a more software-engineering-centric approach: its context retrieval is program-structure aware, and when tests are available it can integrate **SBFL** to sharpen retrieval. That is highly relevant because it shows a concrete bridge from classical debugging signals into LLM issue resolution. Even so, AutoCodeRover's paper uses fault localization scores rather than interactive debugger state, so it belongs in the **dynamic-analysis-assisted but not debugger-assisted** category. [41]

Agentless is a useful counterpoint because it deliberately argues that complex agent structures may be unnecessary for many issue-resolution tasks. It uses a hierarchical localization pipeline, repair sampling, and patch validation with reproduction tests, achieving strong SWE-bench Lite performance while remaining simpler than many autonomous agent frameworks. This makes it extremely relevant as a baseline for the current project: if a debugger-integrated system cannot beat strong non-agentic or lightly agentic baselines under matched conditions, the debugger integration is not yet justified scientifically. [42]

True **debugger-assisted** agentic debugging remains rare. The clearest established example is **ChatDBG**, which lets the LLM issue actual debugger commands and inspect runtime state. Recent work such as **debug-gym** provides an environment with Pdb tools for developing and evaluating such agents, while **Debug2Fix** argues that debuggers have "surprisingly not made their way into coding agents" and reports gains from integrating Java and Python debuggers via a specialized debug subagent. Those two works are especially important because they move the field toward controlled evaluation of dynamic information seeking rather than only repository-shell interaction. [43]

14. Multi-Agent Debugging

The literature does **not** justify the blanket claim that multi-agent architectures are always better. There are credible successes: **AgentFL** and **LLM4FL** report stronger repository-level fault localization with multi-agent decomposition, and **OpenHands** shows a broader multi-agent and multi-benchmark platform. But the same literature also identifies token overhead, coordination cost, and complexity-management issues as key challenges. The **Agentless** thesis is especially important here because it argues, and empirically supports, that simpler decompositions can outperform or rival more agentic systems on SWE tasks. [44]

For the target project, the evidence therefore supports a conservative stance: use multiple roles only when they correspond to clearly separable evidence-processing functions, such as localization, debugging-state collection, patch proposal, and validation. A monolithic controller plus deterministic tools may be scientifically cleaner for an initial prototype than a large committee of agents. [45]

15. Static vs Dynamic Debugging Taxonomy

The literature supports the following taxonomy.

Category	Typical inputs	Runtime-state access	New information gained	Main risks
Static code analysis	Source only	No	Syntax, control/data structure, patterns	Misses execution-specific faults
LLM static code reasoning	Source, comments, docs	No	Semantic priors, likely fix patterns	Hallucinated explanations
Error-text-assisted static analysis	Source + stack trace or error text	Limited	Failure symptom and coarse location	Proximate-cause bias
Test-feedback iterative repair	Source + failing/passing tests	Indirect	Behavioral feedback and regression checks	Patch overfitting; trial-and-error loops
Execution-trace-assisted analysis	Source + trace / coverage / spectra	Partial	Dynamic path and suspiciousness signals	Weak causal grounding
Runtime-state-assisted debugging	Source + exceptions + variables + frames	Yes	Concrete values, path conditions, live state	Tooling complexity, reproduction dependence
Interactive debugger-assisted debugging	Live debugger control	Yes, interactive	Hypothesis-driven inspection and experiments	Security, latency, agent misuse

Category	Typical inputs	Runtime-state access	New information gained	Main risks
Autonomous debugger control	Full reasoning-action-observation loop over debugger	Yes, high	Potentially closest to expert debugging	Unreliable planning, unsafe actions, higher cost

ChatDBG belongs in the last two rows. **debug-gym** is primarily an environment for those rows. **SWE-Agent**, **OpenHands**, **AutoCodeRover**, and **Agentless** mostly sit in the middle rows: repository-aware, tool-using, and often test-feedback-driven, but usually not debugger-controlling. **LLMAO** sits higher up as test-free, static or minimally dynamic FL. **AutoFL** and **AgentFL** are between execution-assisted FL and repository-level tool-using diagnosis. [46]

16. System Case Studies

16.1 ChatDBG

Official name: ChatDBG: Augmenting Debugging with Large Language Models.
Authors: Kyla H. Levin, Stephen N. Freund, Emery D. Berger, Nicolas Van Kempen.
Organization: UMass Amherst, Williams College, related collaborators; official repo under plasma-umass.
Publication status: FSE 2025 paper; open-source repository.
Goal: Answer natural-language debugging questions by letting an LLM control an underlying debugger. [47]

Architecture and tools. ChatDBG integrates with **Pdb**, **LLDB**, and partially **GDB/WinDBG**. A key design feature is that the LLM can "take the wheel" through function calls that run debugger commands, obtain source or symbol information, inspect values, and navigate state. For Python it also supports an IPython/Jupyter slicing command via ipyflow. This is the strongest current evidence of a true **interactive debugger-controlling agent**. [48]

Interaction loop. The loop is reasoning-action-observation: the model receives an enriched stack and instructions, issues debugger commands, observes returned results, and then answers or proposes follow-up steps. The paper's evaluation explicitly compares enriched stacks and "take the wheel" capability, reporting that enriched stack traces alone help only modestly, while debugger control materially improves results. [49]

Evaluation. On unpublished Python scripts and notebooks, a single specialized query reportedly led to an actionable bug fix **67%** of the time, and one additional dialog step increased that to **85%**. The paper also discusses C/C++ cases. These are case-study style results, not SWE-bench-style repository issue-resolution scores. [50]

Limitations and relevance. The evaluation scope is smaller and less benchmark-standard than repository-level issue benchmarks; the system also raises security concerns because debugger commands can execute code, which the authors mitigate with sanitization and whitelisting. Even so, ChatDBG is arguably the most directly relevant prior work for an agentic debugger project because it demonstrates the precise capability gap the project cares about: interactive state inspection through standard debuggers. [47]

16.2 SWE-Agent

Official name: SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering.
Authors: John Yang and many collaborators.
Organization: Princeton and Stanford groups; official repository under SWE-agent.
Publication status: NeurIPS 2024 paper; open-source repository. [51]

Architecture and tools. SWE-agent emphasizes the **agent-computer interface**: LM-friendly commands for navigating repositories, viewing and editing files, and receiving environment feedback. It is fundamentally a tool-using repository agent operating through a shell-like interface, not through an interactive debugger. [52]

Evaluation. The paper reports **12.47%** resolve rate on full SWE-bench and **18.00%** on SWE-bench Lite with GPT-4 Turbo, with pass@k rising as multiple runs are considered. These paper-era scores are lower than many 2025–2026 leaderboard systems, but they were highly influential in establishing the category. [53]

Current repo status. The current official repository states that development effort has shifted to **mini-SWE-agent**, which it says supersedes the original implementation. That matters for reproduction planning: paper replication and current practical usage are no longer perfectly aligned. [54]

Relevance. SWE-agent is essential reading for controller design, tool abstractions, and benchmark harnesses, but it should not be mistaken for a debugger-assisted system. Its strongest transferable idea is not debugging per se; it is interface design for tool-using software agents. [51]

16.3 OpenHands

Official name: OpenHands: An Open Platform for AI Software Developers as Generalist Agents.
Authors: Wang et al. and a large contributor set.
Publication status: ICLR 2025 conference paper; active open-source project with significant repository evolution since publication. [55]

Architecture. The OpenHands paper presents a generalized agent platform with multiple specialized agents and benchmark integrations across software, web, and general assistance tasks. On SWE-bench Lite, the paper reports that **CodeActAgent v1.8** with **Claude 3.5 Sonnet** achieved **26%** resolve rate. [56]

Current repo status. The current repository is no longer just the paper-era agent codebase. It now foregrounds **Agent Canvas**, says the code is moving, and states that the OpenHands Agent and Agent Server source lives elsewhere. This is a notable case where the official repository is valuable for current architecture and deployment status but not a stable mirror of the exact paper artifact. [57]

Debugger integration. Could not verify first-class debugger control in the paper. The platform can run shell commands, code, and benchmark tasks, but paper-read evidence does not justify claiming Pdb/GDB/LLDB-style interactive debugging. **Classification:** strong generalist agent platform; **not verified** as a debugger-assisted coding agent. [58]

16.4 AutoCodeRover

Official name: AutoCodeRover: Autonomous Program Improvement.
Authors: Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, Abhik Roychoudhury.
Organization: National University of Singapore / collaborators; repository under AutoCodeRoverSG.
Publication status: arXiv/open artifact during the period inspected. [59]

Architecture. Two stages: **context retrieval** with code search APIs, then **patch generation**. A key differentiator is program-structure-aware retrieval using AST-level organization rather than plain files, plus optional SBFL when tests are available. [41]

Evaluation. The inspected paper version reports **19%** pass@1 on SWE-bench Lite, **12.42%** on full SWE-bench, and a boost to **22%** on SWE-bench Lite when SBFL is added in the ACR-sbfl setup. It also reports much lower average token cost than SWE-agent in the same comparison. [60]

Current repo status. The official repository reports more recent, higher numbers, including later SWE-bench Lite and Verified scores, and provides experiment-replication documentation. Those claims are valuable implementation metadata, but the paper-era and repo-era scores should not be merged. [61]

Debugger integration. No verified interactive debugger control. The dynamic signal is **SBFL**, not live debugger state. That makes AutoCodeRover highly relevant as a bridge system between static agents and runtime-informed agents, but not yet an interactive debugger agent. [62]

16.5 Agentless

Official name: Agentless: Demystifying LLM-based Software Engineering Agents.
Authors: Chunqiu Steven Xia and collaborators.
Publication status: arXiv preprint inspected; later ACM publication metadata exists, but the preprint was the substantially read source. [63]

Architecture. Agentless deliberately avoids heavyweight agent loops. It uses a **three-phase** process: localization, repair, and patch validation. Localization is hierarchical: files → classes/functions/variables → edit locations. Validation includes repository regression tests and generated **reproduction tests** for reranking and selecting among candidate patches. [42]

Evaluation. The paper reports **32.00%** on SWE-bench Lite at about **$0.70** average cost per issue, outperforming earlier open-source agent-based approaches in that comparison. The paper's ablations attribute a substantial share of the gain to patch-selection and reproduction-test generation. [64]

Relevance. Agentless is one of the most important comparison points for this project because it shows that carefully structured **non-debugger** pipelines can be extremely competitive. Any debugger-centric prototype should therefore be compared directly against Agentless-like baselines to isolate whether debugger access contributes anything beyond localization and test-driven reranking. [65]

16.6 Other Important Systems Found During Research

RepairAgent is important because it reframes APR as an autonomous tool-using agent on **Defects4J**. It reports repairing **164 bugs**, including **39** not fixed by prior techniques, and emphasizes autonomous interleaving of information gathering, repair-ingredient gathering, and validation. This is strong evidence that agentic repair is viable outside SWE-bench, but again it is not a debugger-first system. [66]

AutoFL is an important LLM-based fault-localization system because it lets the model navigate a repository through function calls and asks it to first generate an explanation and then a culprit method. It improves method-level acc@1 over earlier baselines and is notable for explanation-oriented FL. Still, its evaluation is FL-style ranking, not interactive debugging. [67]

AgentFL and **LLM4FL** are important multi-agent repository-level FL systems. AgentFL models comprehension, navigation, and confirmation, while LLM4FL layers group-wise coverage analysis, Graph-RAG navigation, and reviewer reranking. Both strengthen the case that repository-scale FL benefits from structured decomposition. They also confirm that the community is trying to scale diagnosis beyond small contexts. [68]

debug-gym is important not because it is a solved debugging agent, but because it is a research environment with Pdb tooling, repository access, Docker isolation, and benchmark integrations intended explicitly for interactive debugging agents. For experimental design, it is one of the most directly useful recent artifacts. [69]

Debug2Fix is a highly relevant recent prepublication system because it directly argues that shell/test-only coding agents omit rich runtime information and reports substantial gains from integrating Java and Python debuggers through a debug subagent. Because the paper appears prepublication-stage in the inspected copy, those findings should be treated as promising but not yet as settled as the older agent literature. [70]

17. Dataset and Benchmark Review

Core software-bug and issue benchmarks

Dataset	Languages	Task type	Size	Tests	Issues	Bug-fix commits	Stack trace or errors
SWE-bench	Python	Repository-level issue resolution	2,294	Yes	Yes	Yes	Usually iss and failing tests rathe than stack traces
SWE-bench Lite	Python	Lower-cost SWE-bench subset	300	Yes	Yes	Yes	Similar to SWE-bench

Dataset	Languages	Task type	Size	Tests	Issues	Bug-fix commits	Stack trace or errors
SWE-bench Verified	Python	Human-validated SWE benchmark	500	Yes	Yes	Yes	Similar to SWE-bench
BugsInPy	Python	Real bug reproduction/ debugging/ repair	493 bugs, 17 projects	Yes	Partial via metadata/ repo history	Yes	Reproduci failing test not primar stack trace
Defects4J	Java	Real bug reproduction/ debugging/ repair	854 active bugs plus 10 deprecated	Yes	Yes, linked issue metadata	Yes	Triggering tests; not primarily stack trace
QuixBugs	Python, Java	Small algorithmic bug repair	40	Yes	No natural-language issues	No issue discussions; buggy/fixed programs	No
DebugBench	Python, Java, C++	LLM debugging evaluation	4,253	Includes runnable instances and runtime-feedback experiments	No repository issues	Synthetic bug injection	Error and runtime-feedback setting
HumanEvalFix	Multi-language	Function-level bug fixing	164	Yes	No	Synthetic from HumanEval tasks	No
debug-gym integrated tasks	Python-focused environment	Interactive debugging agent evaluation	Environment, not one fixed dataset	Depends on integrated benchmark	Depends	Depends	Yes, via Pd and environme tools

Evidence supporting the core rows is strong. SWE-bench defines real-world issue resolution over repository snapshots and test suites; the official site reports **2,294** full instances, **300** Lite, and **500** Verified. BugsInPy contains **493** real Python bugs from **17** projects. The current Defects4J repository reports **854** active bugs plus **10** deprecated bugs, with fixed single-commit bugs and verified triggering tests. QuixBugs contains **40** programs in both Python and Java with passing and failing tests. DebugBench contains **4,253** instances across three languages. [71]

Practical suitability

For the target project, the most useful datasets divide into three layers. **First**, repository-level issue benchmarks such as SWE-bench measure end-to-end repository reasoning and patch validation, but they remain mostly static or test-feedback-centric. **Second**, real bug corpora such as BugsInPy and Defects4J are better for fault localization, APR, and controlled debugging experiments because they ship reproducible faulty revisions and tests. **Third**, debugging-specific benchmarks and environments such as DebugBench and debug-gym are more appropriate for evaluating debugging behavior itself, especially when runtime interaction or feedback matters. [72]

Leakage and contamination concerns

Contamination and leakage concerns are real. DebugBench was explicitly motivated by leakage risk in prior debugging evaluations. HumanEval-derived tasks also raise decontamination concerns because the original HumanEval benchmark is widely known and relatively small. QuixBugs is useful but has long been a popular APR benchmark, so it is not ideal as a sole modern evaluation target. SWE-bench's realism is higher, but it also reflects widely used public repositories and public issue histories, which complicates strict contamination reasoning for frontier models. [73]

18. Approach Comparison Matrix

Paradigm	Required inputs	Static or dynamic	Runtime-state access	Repo awareness	Can request more evidence	Patch generation	Test validation	Iterative repair
Manual debugging	Code, failure, tools	Dynamic	Yes	Yes	Yes	Human	Human	Yes
Classical automated debugging	Tests, traces, predicates, slices	Dynamic	Partial	Low–medium	Limited	Usually no	N/A or partial	Limited
Fault localization	Tests/ coverage or code features	Usually dynamic	Low–partial	Low–medium	No	No	N/A	No
APR	Fault space + tests	Mixed	Usually indirect	Low–medium	Limited	Yes	Yes	Sometimes
LLM static repair	Code, prompt	Static	No	Variable	No	Yes	Sometimes	Sometimes

Paradigm	Required inputs	Static or dynamic	Runtime-state access	Repo awareness	Can request more evidence	Patch generation	Test validation	Iterative repair
RAG-supported repair	Code + retrieved context	Mostly static	No	High	Limited	Yes	Sometimes	Sometimes
Tool-using repository agent	Repo + shell + tests	Mixed	Indirect	High	Yes	Yes	Yes	Yes
Debugger-assisted agent	Repo + debugger + tests or crash	Dynamic	Yes	High	Yes	Yes	Yes	Yes
Autonomous debugger control	Live debugger + controller	Dynamic	Yes	High	Yes	Yes	Yes	Yes

This matrix summarizes the main conclusion of the review: the scientific novelty opportunity is **not** merely adding more agent steps, but adding a new evidence channel—**live runtime state**—and showing when it helps. [74]

19. System Capability Matrix

System	Static code	Error text	Tests	Execution trace	Runtime variables	Stack frames	Interactive debugger	Repo navigation	Code search	Pa ge
ChatDBG	Yes	Yes	Partial	Partial	Yes	Yes	Yes	Partial	Partial	Ye
SWE-Agent	Yes	Yes	Yes	Partial	No	No	No	Yes	Yes	Ye
OpenHands	Yes	Yes	Yes	Partial	Unclear	Unclear	Unclear	Yes	Yes	Ye
AutoCodeRover	Yes	Yes	Partial	No	No	No	No	Yes	Yes	Ye
Agentless	Yes	Yes	Yes	No	No	No	No	Yes	Yes	Ye
RepairAgent	Yes	Yes	Yes	Partial	No	No	No	Partial	Partial	Ye
AutoFL	Yes	Yes	Yes	Partial	No	No	No	Yes	Yes	No

System	Static code	Error text	Tests	Execution trace	Runtime variables	Stack frames	Interactive debugger	Repo navigation	Code search	Pa ge
AgentFL	Yes	Yes	Yes	Partial	No	No	No	Yes	Yes	No
LLM4FL	Yes	Yes	Yes	Partial	No	No	No	Yes	Yes	No
debug-gym	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Pa
Debug2Fix	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Ye

Values such as **Partial**, **Unclear**, and **Not reported** are deliberate. They reflect exactly the difference between strong paper evidence and inference. For example, OpenHands is unquestionably a rich agent platform, but the inspected paper and current repo do not justify a firm “Yes” on debugger integration. [75]

20. Dataset Suitability Matrix

Dataset	Languages	Task type	Includes tests	Includes issues	Includes bug-fix commits	Includes stack traces or errors	Suitable for SFT	Suitable for RAG	S f e
SWE-bench	Python	Repo issue resolution	Yes	Yes	Yes	Partial	Partial	Yes	Y
SWE-bench Lite	Python	Lower-cost issue resolution	Yes	Yes	Yes	Partial	Partial	Yes	Y
SWE-bench Verified	Python	Human-validated issue resolution	Yes	Yes	Yes	Partial	Partial	Yes	Y
BugsInPy	Python	Real bug reproduction/ repair	Yes	Partial	Yes	Partial	Yes	Yes	Y
Defects4J	Java	Real bug reproduction/ repair	Yes	Yes	Yes	Partial	Yes	Yes	Y

Dataset	Languages	Task type	Includes tests	Includes issues	Includes bug-fix commits	Includes stack traces or errors	Suitable for SFT	Suitable for RAG	S f e
QuixBugs	Python, Java	Small bug repair	Yes	No	Partial	No	Yes	Limited	Y
DebugBench	Python, Java, C++	Debugging ability	Yes or runnable evaluation setup	No	No	Yes	Yes	Limited	Y
HumanEvalFix	Multi-language	Function-level bug fixing	Yes	No	No	No	Yes	Limited	Y
debug-gym	Python-centered env	Interactive debugging	Depends on integrated task	Depends	Depends	Yes	No	No	Y

21. Evaluation Methods and Metrics

Fault localization metrics

The standard FL metrics remain **Top-N**, **EXAM**, and rank-based measures such as **MRR** where applicable. The SBFL survey explains Top-N categories and EXAM clearly, and many modern LLM FL papers still report Top-1, Top-3, or Top-5 rather than developer-centric time-to-fix. That is useful for comparative research, but weak as a sole measure of debugging usefulness. [76]

APR and repository-agent metrics

APR commonly distinguishes **plausible patches** from **correct patches**. Repository-level benchmarks such as SWE-bench simplify the distinction by using test-suite resolution metrics, but the overfitting literature shows that passing available tests is still not a guarantee of correctness. For issue-resolution agents, common metrics are **instances resolved**, cost, token usage, time per task, and pass@k when multiple attempts are allowed. [77]

Root-cause metrics

Evidence insufficient for a single standard code-debugging root-cause metric. The strongest literature-supported options are human annotation against gold explanations, agreement with developer-written fixes or postmortems, and counterfactual checks showing that altering the identified cause removes the failure. OpenRCA offers a concrete RCA benchmark style for operational telemetry, but a widely accepted equivalent for source-code debugger-based RCA has not yet crystallized. [78]

22. Major Findings

1. **The most directly relevant prior work is ChatDBG, not SWE-bench-style agents.** It is the clearest published system that allows an LLM to control a standard debugger and inspect live state. [48]

2. **Most current "agentic debugging" systems are actually repository repair agents.** SWE-Agent, AutoCodeRover, Agentless, and OpenHands are important comparisons, but they usually operate via repository navigation, shell use, test execution, and patching rather than debugger control. [79]

3. **Fault localization is necessary but not sufficient for root-cause analysis.** The literature's dominant FL metrics assess ranking quality, not causal correctness. [27]

4. **APR teaches a hard lesson about validation:** passing tests is often only a plausibility filter, and patch overfitting remains central. [36]

5. **Dynamic signals help, but the field underuses debugger state.** AutoCodeRover's SBFL gains, Debug2Fix's debugger-integration gains, and debug-gym's environment argument all point in that direction. [80]

6. **A Python/Pdb first prototype is strongly supported by available evidence and benchmarks.** BugsInPy, ChatDBG's Python integration, and Pdb's accessibility all favor that path. [81]

23. Contradictions, Uncertainties, and Evidence Strength

The largest contradiction is terminological. Papers and repositories often use the language of "debugging," "issue solving," and "repair" interchangeably, but the underlying evidence channels differ sharply. A shell-and-test loop is not the same as debugger-assisted diagnosis. This report therefore treats many systems that self-describe as debugging agents as **repair agents** unless debugger control or direct runtime-state querying was verified. [82]

A second recurring uncertainty comes from **version drift**. Repository READMEs often advertise results newer than the inspected paper. For example, the AutoCodeRover and Agentless repositories report substantially higher newer scores than their original papers; the SWE-agent repository now recommends mini-SWE-agent; and the OpenHands repo has restructured around Agent Canvas. Those are real facts about project evolution, but they should not be back-projected into claims about the original papers. [83]

A third uncertainty concerns the newest debugger-centric work. Debug2Fix and some 2026 materials are highly relevant, but they are newer and less established than the already peer-reviewed SWE-agent, OpenHands, and ChatDBG literature. The evidence is promising, not definitive. [84]

24. Research Gaps

Well-established gaps

- **Root-cause reasoning vs. suspicious ranking.** Most FL work stops at ranking locations. [85]
- **Patch plausibility vs. semantic correctness.** APR still struggles with overfitting and weak tests. [86]

- **Weak runtime-state use in LLM repair.** Shell and test feedback dominate; debugger state is comparatively rare. [87]
- **Repository navigation vs. dynamic diagnosis.** Current agents are better at file and tool navigation than at principled runtime diagnosis. [88]
- **Evaluation realism and contamination.** DebugBench and newer benchmark analyses explicitly raise leakage and realism concerns. [89]

Inferred research opportunities

- A benchmark or protocol that measures whether **debugger interaction adds value beyond issue text plus tests**.
- Methods that convert raw debugger observations into **causally grounded explanations** rather than only better patches.
- Small or medium open-weight models that rely more on **deterministic debugging tools** and less on broad parametric memorization.
- Portable debugger adapters across **Pdb, GDB, and LLDB** with a shared action schema and safety model.
- Hybrid evaluation combining **location accuracy**, **explanation correctness**, **repair correctness**, and **interaction cost**. [90]

25. Implications for the Agentic Debugging Project

Conclusions directly supported by literature

- **Debugger interaction provides information unavailable to static agents.** This is explicit in ChatDBG, debug-gym, and Debug2Fix. [43]
- **Python/Pdb is a reasonable first prototype target.** ChatDBG supports Pdb; debug-gym is Pdb-based; BugsInPy provides real Python bugs. [91]
- **Deterministic tools should own evidence acquisition.** SWE-Agent and ChatDBG both show that interface design matters; letting the model query tools is more reliable than forcing it to infer missing state. [92]
- **A verifier is necessary.** APR literature and Agentless-style patch-selection results both show the need for independent validation beyond first candidate generation. [93]

Reasonable engineering recommendations

- Put **debugger command execution, test execution, stack extraction, variable serialization, and repository search** in deterministic tool wrappers.
- Put **hypothesis generation, evidence prioritization, causal explanation drafting, and patch proposal** in the model.
- Put **budgeting, loop control, retries, and safety policy** in the agent controller.
- Keep a **separate verifier** that checks whether a proposed explanation is consistent with observed evidence and whether a patch actually repairs the failing behavior without regressions. [94]

Unresolved questions requiring experiments

- How much does debugger state improve over a strong Agentless or AutoCodeRover-style baseline on the same tasks?

- Which runtime observations are most useful: stack frames, locals, watch expressions, step traces, or breakpoints?
- Can a small open-weight model use debugger tools effectively enough, or is a larger model needed at first?
- Does debugger access improve **root-cause explanation quality**, not just patch success? [95]

Suggested MVP

A realistic minimum viable research prototype should contain:

1. **Python-only scope** with **Pdb** integration.
2. A reproducible failure source: failing pytest or exception-triggering script.
3. Deterministic tools for stack frames, locals, expression evaluation, stepping, breakpoints, source snippets, and repository search.
4. A controller loop: reproduce → inspect → hypothesize → gather more evidence → localize → patch → validate → revise once or twice.
5. Evaluation on a curated Python subset, ideally from **BugsInPy**, with a static-agent baseline and an ablation that removes debugger access. [96]

Out of scope initially: multi-language debugger portability, full multi-agent orchestration, paid-API-dependent production workflows, and benchmark pursuit on SWE-bench as the primary scientific objective. The literature does not show that those are necessary for a first publishable debugger-agent result. [45]

26. Recommended Next Research Steps

1. **Read ChatDBG completely and reproduce its debugger action model conceptually.** It is the closest direct prior art. [48]
2. **Read debug-gym and Debug2Fix for interaction design and evaluation framing.** They provide the clearest recent debugger-centric path. [97]
3. **Use Agentless, SWE-Agent, and AutoCodeRover as non-debugger baselines.** They anchor the comparison against strong repository-level systems. [98]
4. **Choose a Python/BugsInPy subset** where failures are reproducible and the debugger meaningfully exposes state. [99]
5. **Design evaluation around evidence contribution**, not only final patch rate: localization accuracy, explanation faithfulness, debugger actions used, extra cost, and ablations without runtime state. [100]

27. Priority Reading List

TIER 1 — Must read completely

- **ChatDBG: Augmenting Debugging with Large Language Models.** Read architecture, prompt design, “take the wheel,” evaluation, and security sections. Most directly relevant work. [48]
- **SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering.** Read ACI design, agent loop, evaluation, and appendix on trajectories/costs. Essential for controller/interface thinking. [40]

- **Agentless: Demystifying LLM-based Software Engineering Agents.** Read localization, repair, validation, and ablations. Essential as a strong simplicity baseline. [65]
- **AutoCodeRover: Autonomous Program Improvement.** Read context retrieval, SBFL integration, and evaluation. Important bridge from static agents toward dynamic signals. [101]
- **debug-gym: A Text-Based Environment for Interactive Debugging.** Read environment design, tools, benchmark integration, and limitations. Essential for experimental setup ideas. [69]
- **Automated Program Repair** CACM overview. Read the full article for APR taxonomy and field lessons. [102]

TIER 2 — Read core sections

- **OpenHands** paper: software engineering sections and runtime workflow. [56]
- **RepairAgent**: introduction, tool architecture, finite-state machine, and evaluation on Defects4J. [66]
- **Large Language Models for Test-Free Fault Localization**: formulation, datasets, and evaluation. [25]
- **AutoFL**: method and explanation-generation stages. [67]
- **Scalable Statistical Bug Isolation** and **Whyline**: for diagnosis and explanation foundations. [103]

TIER 3 — Useful supporting source

- **BugsInPy** paper and repo. [99]
- **Defects4J** paper/repo and current metadata. [104]
- **QuixBugs** paper and repo. [105]
- **ChatRepair** for test-feedback conversational APR. [106]
- **InferFix** for retrieval-augmented static-analysis-driven repair. [35]
- **AgentFL** and **LLM4FL** for multi-agent FL baselines. [68]

TIER 4 — Background or optional

- **Causal reasoning in Software Quality Assurance**. Useful for framing RCA rigor. [107]
- **OpenRCA**. Useful for RCA evaluation ideas outside code debugging. [108]
- **DebugBench**. Useful for broader debugging-evaluation context. [109]
- **Debug2Fix**. Very relevant, but newer and less settled; read after foundational material. [70]

28. Full Source Inventory

A. Source Ledger

Source ID	Full title	Authors	Year	Venue	Source type	Peer reviewed?	Access level
S01	ChatDBG: Augmenting Debugging with Large Language Models	Levin et al.	2025	FSE	Paper	Yes	SUBSTANTIAL_

Source ID	Full title	Authors	Year	Venue	Source type	Peer reviewed?	Access level
S02	ChatDBG GitHub repository	plasma-umass	2026 checked	GitHub	Repository	No	SUBSTANTIAL_
S03	SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering	Yang et al.	2024	NeurIPS	Paper	Yes	SUBSTANTIAL_
S04	SWE-agent GitHub repository	SWE-agent team	2026 checked	GitHub	Repository	No	SUBSTANTIAL_
S05	OpenHands: An Open Platform for AI Software Developers as Generalist Agents	Wang et al.	2025	ICLR	Paper	Yes	SUBSTANTIAL_
S06	OpenHands GitHub repository	OpenHands	2026 checked	GitHub	Repository	No	SUBSTANTIAL_
S07	AutoCodeRover: Autonomous Program Improvement	Zhang et al.	2024	arXiv	Preprint	No	SUBSTANTIAL_
S08	AutoCodeRover GitHub repository	AutoCodeRoverSG	2026 checked	GitHub	Repository	No	SUBSTANTIAL_
S09	Agentless: Demystifying LLM-based Software Engineering Agents	Xia et al.	2024	arXiv	Preprint	No	SUBSTANTIAL_

Source ID	Full title	Authors	Year	Venue	Source type	Peer reviewed?	Access level
S10	Agentless GitHub repository	OpenAutoCoder	2026 checked	GitHub	Repository	No	SUBSTANTIAL_
S11	RepairAgent: An Autonomous, LLM-Based Agent for Program Repair	Bouzenia et al.	2025	ICSE	Paper	Yes	SUBSTANTIAL_
S12	InferFix: End-to-End Program Repair with LLMs over Retrieval-Augmented Prompts	Jin et al.	2023	Industry paper / ACM metadata	Mixed	Could not verify final venue from inspected sections	SUBSTANTIAL_
S13	Automated Program Repair via Conversation	Xia and Zhang	2024	ISSTA	Paper	Yes	SUBSTANTIAL_
S14	Automated Program Repair	Le Goues, Pradel, Roychoudhury	2019	CACM	Article	Yes	SUBSTANTIAL_
S15	SemFix: Program Repair via Semantic Analysis	Nguyen et al.	2013	ICSE	Paper	Yes	METADATA_ON
S16	PAR: Automatic Patch Generation Learned from Human-Written Patches	Kim et al.	2013	ICSE	Paper	Yes	METADATA_ON

Source ID	Full title	Authors	Year	Venue	Source type	Peer reviewed?	Access level
S17	A Critical Review of Automatic Patch Generation Learned from Human-Written Patches	Monperrus	2014	ICSE	Paper	Yes	ABSTRACT_ON
S18	An Analysis of Patch Plausibility and Correctness for Generate-and-Validate Patch Generation Systems	Qi et al.	2015	ISSTA	Paper	Yes	ABSTRACT_ON
S19	Debugging Reinvented: Asking and Answering Why and Why Not Questions about Program Behavior	Ko and Myers	2008	ICSE/PLDI-era HCI/SE paper context	Paper	Yes	SUBSTANTIAL_
S20	Scalable Statistical Bug Isolation	Liblit et al.	2005	PLDI	Paper	Yes	SUBSTANTIAL_
S21	A Grounded Theory of Debugging in Professional Software Engineering Practice	Li and Coblenz	2026	Proc. ACM Softw. Eng. FSE	Paper	Yes	SUBSTANTIAL_
S22	Dynamically Discovering Likely Program Invariants	Ernst et al.	2001	IEEE TSE	Paper	Yes	ABSTRACT_ON

Source ID	Full title	Authors	Year	Venue	Source type	Peer reviewed?	Access level
S23	A Survey of Challenges in Spectrum-Based Software Fault Localization	Sarhan and Beszédes	2022	IEEE Access	Survey	Yes	SUBSTANTIAL_
S24	Large Language Models for Test-Free Fault Localization	Yang et al.	2024	ICSE	Paper	Yes	SUBSTANTIAL_
S25	A Quantitative and Qualitative Evaluation of LLM-Based Explainable Fault Localization	Kang et al.	2024	TOSEM / ACM	Paper	Yes	SUBSTANTIAL_
S26	AgentFL: Scaling LLM-based Fault Localization to Project-Level Context	Qin et al.	2024	arXiv	Preprint	No	ABSTRACT_ON
S27	LLM4FL: A Multi-Agent Approach to Fault Localization via Graph-Based Retrieval and Reflexion	Rafi et al.	2025	arXiv/ OpenReview	Preprint	No	SUBSTANTIAL_
S28	OpenRCA: Can Large Language Models Locate the Root Cause of Software Failures?	Xu et al.	2025	OpenReview/ ICLR workshop context	Mixed	Mixed	ABSTRACT_ON

Source ID	Full title	Authors	Year	Venue	Source type	Peer reviewed?	Access level
S29	Causal Reasoning in Software Quality Assurance: A Systematic Review	Giamattei et al.	2025	Information and Software Technology	Survey	Yes	ABSTRACT_ON
S30	debug-gym: A Text-Based Environment for Interactive Debugging	Yuan et al.	2025	Technical report / arXiv	Preprint	No	SUBSTANTIAL_
S31	Debug2Fix: Supercharging Coding Agents with Interactive Debugging Capabilities	Garg and Huang	2026	Prepublication PDF	Unclear	No	SUBSTANTIAL_
S32	SWE-bench paper	Jimenez et al.	2024	ICLR	Paper	Yes	SUBSTANTIAL_
S33	SWE-bench official FAQ and benchmark pages	SWE-bench team	2026 checked	Official website	Benchmark docs	No	SUBSTANTIAL_
S34	Defects4J official repository	Just et al. maintainers	2026 checked	GitHub	Repository	No	SUBSTANTIAL_
S35	BugsInPy paper	Widyasari et al.	2024	Paper mirror / associated with ISSTA 2020 line	Mixed	Mixed	SUBSTANTIAL_
S36	BugsInPy official repository	soarsmu	2026 checked	GitHub	Repository	No	SUBSTANTIAL_
S37	QuixBugs paper	Lin et al.	2017	Workshop/ paper PDF	Paper	Yes	SUBSTANTIAL_

Source ID	Full title	Authors	Year	Venue	Source type	Peer reviewed?	Access level
S38	QuixBugs official repository	jkoppel	2026 checked	GitHub	Repository	No	METADATA_ON
S39	DebugBench paper	Tian et al.	2024	ACL Findings	Paper	Yes	SUBSTANTIAL_
S40	DebugBench official repository	THUNLP	2026 checked	GitHub	Repository	No	METADATA_ON

B. Paper-by-Paper Evidence Table

Source ID	Research question	Method	Dataset or benchmark	System or model	Main findings	Reported metrics	Limita stated autho
S01	Can LLMs meaningfully augment debuggers?	LLM + debugger function calls	Python scripts, notebooks, C/C++ cases	ChatDBG	Debugger control improves diagnosis and fixes	57%, 67%, 85% in Python dialog settings	Small evalu
S03	Does ACI design help software agents?	LM + ACI + shell tools	SWE-bench, HumanEvalFix	SWE-agent	Interface design matters strongly	12.47% full, 18.00% Lite	Exper brittle trajec
S05	Can a broad platform support many agents and tasks?	Multi-agent platform	SWE-bench Lite, WebArena, GAIA, etc.	OpenHands	Competitive generalist software-agent capabilities	26% on SWE-bench Lite with Claude 3.5 Sonnet	Broad bench deper
S07	Can structure-aware retrieval improve repo issue solving?	AST-aware retrieval + patch generation + optional SBFL	SWE-bench Lite/full	AutoCodeRover	Strong cost-efficiency; SBFL helps	19% Lite, 22% with SBFL	Limite paper score
S09	Are heavy agents necessary?	Hierarchical localization + repair + validation	SWE-bench Lite	Agentless	Simple architecture can be highly competitive	32% Lite	Repro tests

Source ID	Research question	Method	Dataset or benchmark	System or model	Main findings	Reported metrics	Limita stated autho
S11	Can APR be performed by an autonomous LLM agent?	Tool-using LLM + FSM controller	Defects4J	RepairAgent	Agentic APR can repair many bugs	164 bugs repaired	Cost a desig trade
S12	Can retrieval and static analysis improve LLM repair?	Static analyzer + retriever + finetuned generator	InferredBugs	InferFix	Strong top-1 fix accuracy for static-analysis-found bugs	65.6% C#, 76.8% Java	Doma specif sourc
S13	Does conversational feedback help APR?	Interleaved patch generation and test feedback	337 APR bugs	ChatRepair	Conversation improves repair efficiency/ effectiveness	162/337 bugs for $0.42 each	Test-s depe
S23	What are SBFL challenges and metrics?	Survey	FL literature	Survey	Summarizes formulas, ties, Top-N, EXAM issues	N/A	Surve limite SBFL
S24	Can LLMs localize faults without tests?	Finetuned LLM representations	Defects4J and bug corpora	LLMAO	LLM-based FL can work without coverage info	Top-1 and Top-5 gains over MLFL baselines	Still locali only
S25	Can LLMs do explainable repository-level FL?	Tool-using LLM with explanations	798 Java/ Python bugs	AutoFL	Explanation-oriented FL can improve acc@1	up to 233.3% acc@1 improvement over baselines	Meth focus
S30	How should interactive debugging environments for agents be built?	Environment/ tool framework	Aider, Mini-nightmare, SWE-bench integrations	debug-gym	Provides Pdb-enabled environment for research	Environment-oriented	Not p best- perfo agent

Source ID	Research question	Method	Dataset or benchmark	System or model	Main findings	Reported metrics	Limita stated autho
S31	Does debugger integration help coding agents?	Debug subagent + JDB/PDB	GitBug-Java, SWE-Bench-Live	Debug2Fix	Reports >20% relative improvement for some models	Relative improvement claim	Prepu stage

29. Inaccessible or Partially Unread Sources

A. Papers I could not access fully

1. **SemFix: Program Repair via Semantic Analysis** — Nguyen, Qi, Roychoudhury, Chandra, 2013, ICSE. Relevant as a seminal semantic APR system. A PDF landing page was found, but only metadata/snippet-level inspection was used in this report.
2. **PAR: Automatic Patch Generation Learned from Human-Written Patches** — Kim et al., 2013, ICSE. Relevant as a seminal template-based repair system. Only metadata/snippet-level inspection was used.
3. **AgentFL: Scaling LLM-based Fault Localization to Project-Level Context** — Qin et al., 2024, arXiv. Abstract available and used cautiously for high-level positioning, but not fully read.
4. **OpenRCA: Can Large Language Models Locate the Root Cause of Software Failures?** — Xu et al., 2025. Abstract and metadata available; full code-debugging-relevant details were not relied upon.
5. **Causal Reasoning in Software Quality Assurance: A Systematic Review** — Giamattei et al., 2025. Abstract inspected; not fully read in this review cycle.

B. Claims I could not verify

- Exact historical publication venue/version metadata for some benchmark papers where only repository/artifact sources were substantially read.
- Current license text for **AutoCodeRover** from the inspected repository page.
- Whether current **OpenHands** product variants support first-class debugger integration in a way comparable to ChatDBG.
- A standardized, widely accepted metric for code-level root-cause correctness analogous to Top-N for FL.

C. Questions that require experiments rather than literature review

- Does debugger access produce statistically significant gains over Agentless/AutoCodeRover baselines on matched Python bug subsets?
- Which debugger actions are most useful for the model under budget constraints?
- How well can small or medium open-weight code models use a debugger compared with larger closed models?
- Do debugger-observed states improve explanation faithfulness or only patch success?

30. PDF Download Manifest

Priority	Suggested filename	Full title
High	2025_chatdbg_augmenting_debugging_with_large_language_models.pdf	ChatDBG: Augmenting Debugging with Large Language Models
High	2024_swe_agent_agent_computer_interfaces_enable_automated_software_engineering.pdf	SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering
High	2025_openhands_open_platform_for_ai_software_developers.pdf	OpenHands
High	2024_autocoderover_autonomous_program_improvement.pdf	AutoCodeRover: Autonomous Program Improvement
High	2024_agentless_demystifying_llm_based_software_engineering_agents.pdf	Agentless: Demystifying LLM-based Software Engineering Agents
High	2025_repairagent_an_autonomous_llm_based_agent_for_program_repair.pdf	RepairAgent
High	2025_debug_gym_text_based_environment_for_interactive_debugging.pdf	debug-gym
High	2026_debug2fix_supercharging_coding_agents_with_interactive_debugging_capabilities.pdf	Debug2Fix

Priority	Suggested filename	Full title
High	2019_automated_program_repair_cacm.pdf	Automated Program Repair
High	2024_llms_for_test_free_fault_localization.pdf	Large Language Models for Test-Free Fault Localization
High	2024_autofl_explainable_fault_localization.pdf	A Quantitative and Qualitative Evaluation of LLM-Based Explainable Fault Localization
High	2020_2024_bugsinpy_database_of_existing_bugs_in_python_programs.pdf	BugsInPy
High	2017_quixbugs_multilingual_program_repair_benchmark.pdf	QuixBugs
Medium	2005_scalable_statistical_bug_isolation.pdf	Scalable Statistical Bug Isolation
Medium	2008_debugging_reinvented_whyline.pdf	Debugging Reinvented
Medium	2022_survey_of_challenges_in_spectrum_based_fault_localization.pdf	A Survey of Challenges in Spectrum-Based Software Fault Localization
Medium	2024_debugbench_evaluating_debugging_capability_of_llms.pdf	DebugBench

31. Claim-Evidence Matrix

F. Evidence Strength Matrix

Claim ID	Claim	Supporting sources	Contradicting sources	Evidence strength	Confidence	Notes
C01	ChatDBG is the clearest published debugger-controlling LLM system directly relevant to this project	S01, S02	None found	Strong	High	Verified debugger integration
C02	Most SWE-style agents are repository repair agents, not interactive debugger agents	S03, S05, S07, S09	None found	Strong	High	Tool use is mostly shell/tests
C03	Fault localization metrics do not by themselves prove root-cause understanding	S19, S20, S23, S25	None found	Strong	High	Ranking vs causal explanation distinction
C04	Test-suite-based APR remains vulnerable to overfitting	S14, S17, S18	None found	Strong	High	Canonical APR lesson
C05	Dynamic debugger state is underused in modern LLM repair systems	S01, S30, S31	None found	Moderate	Medium-high	Recent debugger-centric work still sparse
C06	Python/Pdb is the best-evidenced first prototype path	S01, S30, S35, S36	No strong contrary evidence	Moderate	Medium-high	Strong practical and benchmark support

Claim ID	Claim	Supporting sources	Contradicting sources	Evidence strength	Confidence	Notes
C07	Multi-agent design is not automatically superior	S09, S26, S27, S05	None found	Moderate	Medium	Evidence mostly indirect and comparative
C08	A scientifically distinct contribution would be demonstrating benefit from debugger interaction over strong static-agent baselines	S01, S07, S09, S30, S31	None found	Moderate	Medium-high	Requires experiments for confirmation

G. Project-Relevance Matrix

Prior work or technique	Potentially reusable component	Required adaptation	Main risk	Expected benefit	Evidence level	Recommended priority
ChatDBG	Debugger action schema, natural-language question framing	Modern controller, safer tool wrappers, stronger evaluation	Small evaluation scope	Direct blueprint for debugger integration	Strong	Highest
SWE-Agent	ACI/controller abstractions	Replace shell-first bias with debugger-first evidence collection	Expensive trajectory loops	Robust orchestration ideas	Strong	High
AutoCodeRover	Structure-aware retrieval and optional SBFL	Integrate with debugger evidence instead of only retrieval	Still not runtime-state centric	Better repository context targeting	Strong	High

Prior work or technique	Potentially reusable component	Required adaptation	Main risk	Expected benefit	Evidence level	Recommended priority
Agentless	Hierarchical localization and validation discipline	Add debugger-state phase and stronger explanation checks	May already solve easy cases without debugger	Powerful baseline and patch-selection lessons	Strong	Highest
RepairAgent	Agentic APR loop and tool FSM ideas	Add repository scale and debugger tools	Java/ Defects4J framing differs	Helpful for repair-controller design	Moderate	Medium-high
debug-gym	Interactive environment concepts	Adapt to project-specific tooling and evaluation	Environment maturity	Faster experimentation on debugger agents	Strong	Highest
Debug2Fix	Debug subagent concept	Reproduce independently, especially on Python	Early-stage evidence	Concrete design pattern for debugger integration	Moderate	High
AutoFL / AgentFL / LLM4FL	Repository-scale FL decomposition	Move from ranking to causal/stateful diagnosis	May remain localization-only	Better localization scaffolding	Moderate	Medium

32. Research Self-Audit

- **Total sources discovered:** 40 materially logged sources in the ledger above.
- **Total unique works after deduplication:** 40 in the ledger; near-duplicates between papers and repositories were intentionally kept separate because they supported different claim types.
- **Total full-text papers read:** 0 claimed as FULL_TEXT_READ.
- **Total substantially read:** 26.
- **Total abstract-only:** 6.
- **Total metadata-only:** 8.
- **Total inaccessible:** 0 strictly inaccessible, but several relevant works were only partially accessed and are flagged accordingly.
- **Total peer-reviewed sources:** 21.
- **Total preprints or non-peer-reviewed artifacts:** 19.
- **Foundational versus recent coverage:** Foundational coverage is strong for debugging, FL, and APR; recent coverage is strong for LLM repair and agents; code-level RCA remains thinner.
- **Whether every major factual claim has a citation:** Yes, for all major claims in the prose above.
- **Whether all inaccessible or partially unread sources are clearly labeled:** Yes.

- **Whether any paper was summarized beyond actually accessed sections:** No intentionally; where only abstract or metadata was accessed, claims were limited accordingly.
- **Whether duplicate publication versions were merged correctly:** Best effort; where paper and repository served different evidentiary functions, both were preserved.
- **Whether system capability claims were verified:** Yes, to the extent possible from inspected papers and official artifacts; otherwise marked Partial, Unclear, or Not reported.
- **Whether contradictory evidence was included:** Yes, especially around repo-versus-paper drift and multi-agent claims.
- **Whether direct PDF links were actually verified:** Only where explicitly indicated as verified in the manifest.
- **Exact final search date:** 2026-07-18.

Verdict: LITERATURE_COVERAGE_ADEQUATE_WITH_GAPS

The coverage is strong enough to guide a first research prototype and to compare against non-debugger baselines, but it still has gaps in code-level root-cause analysis and in fully settled evidence for the newest debugger-agent papers. The literature clearly supports a debugger-centric research direction; it does not yet offer a fully mature evaluation standard for proving debugger-based causal diagnosis.

---

1 4 6 9 43 46 47 48 49 50 74 75 90 91 94 96 https://arxiv.org/html/2403.16354v3
https://arxiv.org/html/2403.16354v3

2 7 10 12 15 28 30 https://faculty.washington.edu/ajko/papers/Ko2008JavaWhyline.pdf
https://faculty.washington.edu/ajko/papers/Ko2008JavaWhyline.pdf

3 11 41 59 60 62 80 101 https://arxiv.org/html/2404.05427v3
https://arxiv.org/html/2404.05427v3

5 29 107 https://arxiv.org/abs/2408.17183
https://arxiv.org/abs/2408.17183

8 54 https://github.com/swe-agent/swe-agent
https://github.com/swe-agent/swe-agent

13 31 32 36 77 86 93 102 https://abhikrc.com/pdf/cacm19.pdf
https://abhikrc.com/pdf/cacm19.pdf

14 16 https://arxiv.org/pdf/2602.11435
https://arxiv.org/pdf/2602.11435

17 18 21 103 https://theory.stanford.edu/~aiken/publications/papers/pldi05.pdf
https://theory.stanford.edu/~aiken/publications/papers/pldi05.pdf

19 20 27 76 85 100 https://publicatio.bibl.u-szeged.hu/39015/1/A_Survey_of_Challenges_in_Spectrum-Based_Software_Fault_Localization-1.pdf
https://publicatio.bibl.u-szeged.hu/39015/1/A_Survey_of_Challenges_in_Spectrum-Based_Software_Fault_Localization-1.pdf

22 https://mboehme.github.io/paper/EMSE21.pdf
https://mboehme.github.io/paper/EMSE21.pdf

23 https://coinse.github.io/publications/pdfs/Moon2014ly.pdf
https://coinse.github.io/publications/pdfs/Moon2014ly.pdf

24 https://homes.cs.washington.edu/~mernst/pubs/invariants-tse2001.pdf
https://homes.cs.washington.edu/~mernst/pubs/invariants-tse2001.pdf

25 38 https://aidanby.github.io/files/icse24.pdf
https://aidanby.github.io/files/icse24.pdf

26 67 https://arxiv.org/html/2308.05487v3
https://arxiv.org/html/2308.05487v3

33 https://people.csail.mit.edu/hunkim/papers/kim-icse2013.pdf
https://people.csail.mit.edu/hunkim/papers/kim-icse2013.pdf

34 https://abhikrc.com/pdf/ICSE13-SEMFIX.pdf
https://abhikrc.com/pdf/ICSE13-SEMFIX.pdf

35 https://arxiv.org/pdf/2303.07263
https://arxiv.org/pdf/2303.07263

37 106 https://lingming.cs.illinois.edu/publications/issta2024.pdf
https://lingming.cs.illinois.edu/publications/issta2024.pdf

39 73 89 https://arxiv.org/abs/2401.04621
https://arxiv.org/abs/2401.04621

40 51 52 53 79 82 88 92 https://proceedings.neurips.cc/paper_files/paper/2024/file/5a7c947568c1b1328ccc5230172e1e7c-Paper-Conference.pdf
https://proceedings.neurips.cc/paper_files/paper/2024/file/5a7c947568c1b1328ccc5230172e1e7c-Paper-Conference.pdf

42 45 63 64 65 95 98 https://arxiv.org/pdf/2407.01489
https://arxiv.org/pdf/2407.01489

44 68 https://arxiv.org/abs/2403.16362
https://arxiv.org/abs/2403.16362

55 56 58 https://proceedings.iclr.cc/paper_files/paper/2025/file/a4b6ad6b48850c0c331d1259fc66a69c-Paper-Conference.pdf
https://proceedings.iclr.cc/paper_files/paper/2025/file/a4b6ad6b48850c0c331d1259fc66a69c-Paper-Conference.pdf

57 https://github.com/OpenHands/openhands
https://github.com/OpenHands/openhands

61 83 https://github.com/AutoCodeRoverSG/auto-code-rover
https://github.com/AutoCodeRoverSG/auto-code-rover

66 https://software-lab.org/publications/icse2025_RepairAgent.pdf
https://software-lab.org/publications/icse2025_RepairAgent.pdf

69 97 https://arxiv.org/pdf/2503.21557
https://arxiv.org/pdf/2503.21557

70 84 87 https://www.microsoft.com/en-us/research/wp-content/uploads/2026/02/cngtkgvhtjjshgbmtcqnsjddkdzdhvyr.pdf
https://www.microsoft.com/en-us/research/wp-content/uploads/2026/02/cngtkgvhtjjshgbmtcqnsjddkdzdhvyr.pdf

71 72 https://www.swebench.com/SWE-bench/faq/
https://www.swebench.com/SWE-bench/faq/

78 108 https://netman.aiops.org/wp-content/uploads/2025/05/13411_OpenRCA_Can_Large_Langua.pdf
https://netman.aiops.org/wp-content/uploads/2025/05/13411_OpenRCA_Can_Large_Langua.pdf

81 99 https://arxiv.org/pdf/2401.15481
https://arxiv.org/pdf/2401.15481

104 https://github.com/rjust/defects4j
https://github.com/rjust/defects4j

105 https://jkoppel.github.io/QuixBugs/quixbugs.pdf
https://jkoppel.github.io/QuixBugs/quixbugs.pdf

109 https://aclanthology.org/2024.findings-acl.247.pdf
https://aclanthology.org/2024.findings-acl.247.pdf